

MC906 · TP3 DEEP LEARNING

Classificação de câncer de mama com MLP em NumPy

Breast Cancer Wisconsin · forward & backprop do zero · interpretabilidade

Adriano Ribeiro Franulovic Campos · Andrei Quaresma Pinto · Matheus Ferracciú Scatolin

Pietro Fernandes Magaldi · Tiago Perrupato Antunes

Prof. Anderson Rocha

Agenda · ~25 min

5 blocos · ~5 slides por apresentador

- | | | |
|----|------------------------------------|---|
| 01 | Contexto & dados | FNA, dataset, pré-processamento, PCA |
| 02 | Implementação | MLP, ReLU, Softmax, CE, backprop |
| 03 | Experimentos | Baseline, hiperparâmetros, early stopping |
| 04 | Resultados & interpret. | Teste, ablação Δ CE, casos individuais |
| 05 | Extras & discussão | He, momentum, sklearn, SHAP, limitações |

BLOCO 1

Problema & objetivos do TP

Application 1: classificação tabular

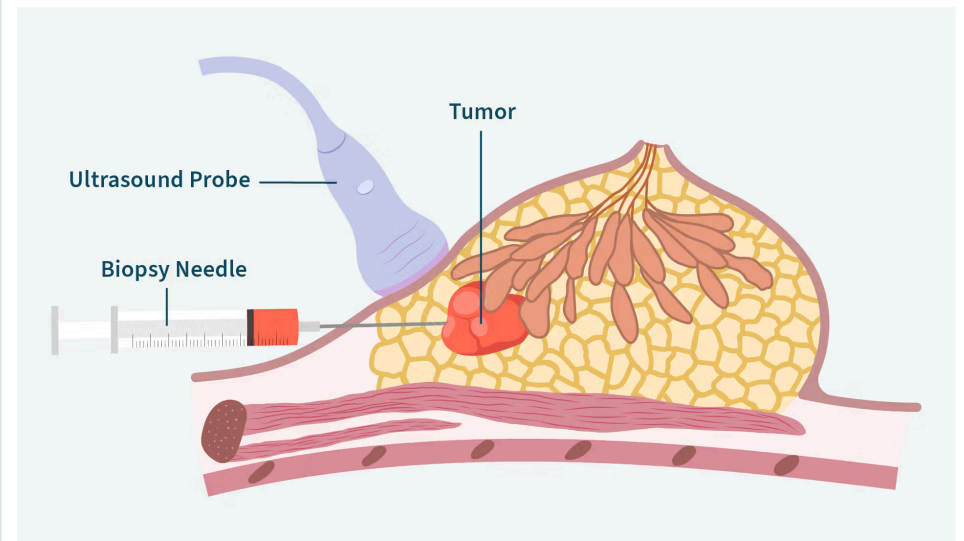
- **Task:** classificar núcleos de aspirado (FNA) como **benigno (B)** ou **maligno (M)**
- **Requisito:** MLP 100% manual com forward, backprop e mini-batch SGD (NumPy only)
- **Ativações:** ReLU (hidden) · Softmax (output) · loss = cross-entropy
- **Entregáveis:** curvas de treino, exploração de hiperparâmetros, interpretabilidade obrigatória
- **Notebook:** `breast-cancer-nn.ipynb`, autocontido, sem PyTorch/TensorFlow

BLOCO 1

Coleta da amostra · Fine Needle Aspiration (FNA)

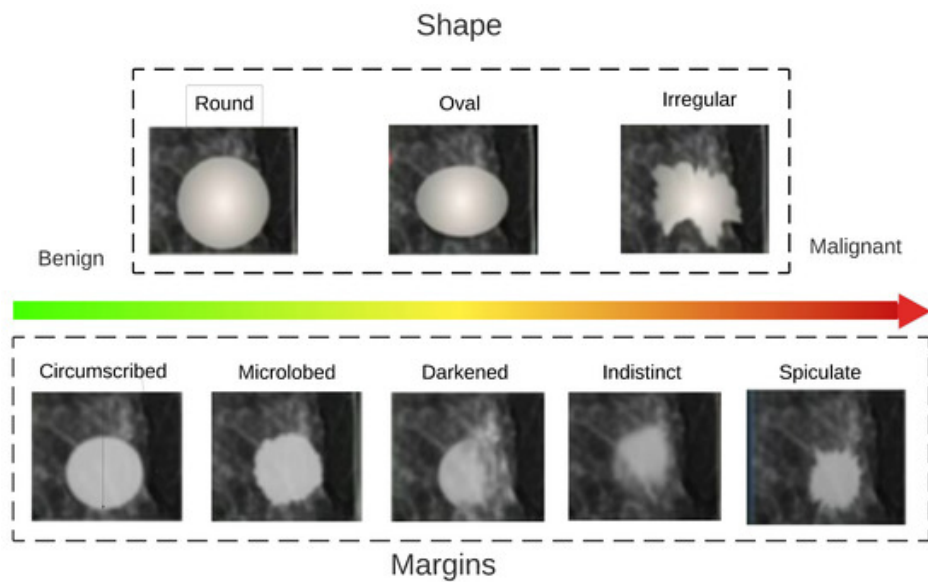
TAKEAWAY

- Agulha fina extrai células da lesão; a lâmina citológica é digitalizada
- O modelo não vê a imagem: usa **30 medidas morfológicas do núcleo**
- Cada feature existe em 3 escalas: `mean_*`, `worst_*`, `*_error`



BLOCO 1

Benigno vs maligno · morfologia celular



TAKEAWAY

- **Benigno:** núcleos uniformes, contornos regulares; valores tendem a ser menores
- **Maligno:** irregularidade e concavidades; `worst_*` tendem a subir
- O MLP aprende correlações estatísticas, não “vê” células como em CNN

B = 0

M = 1

BLOCO 1

Dataset & pré-processamento

Breast Cancer Wisconsin (Diagnostic)

569

AMOSTRAS

30

FEATURES

62.7%

CLASSE B

64%/16%/20%

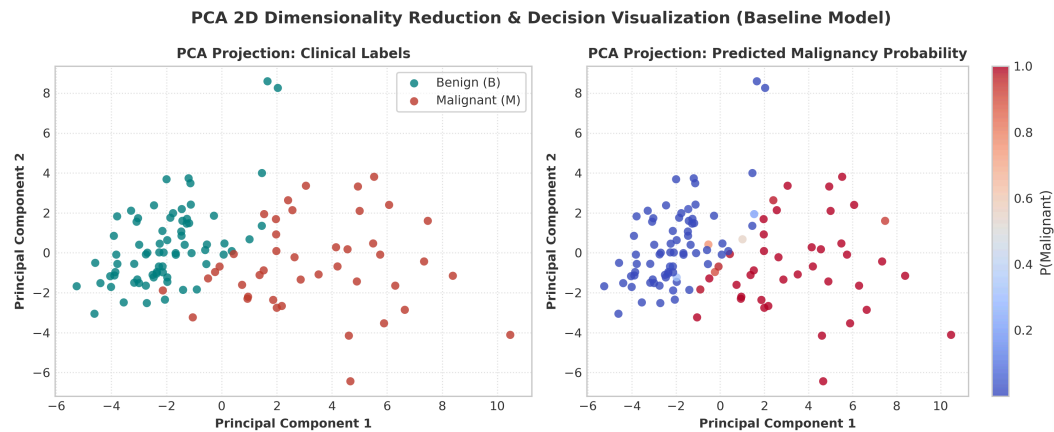
363/91/115

PIPELINE

- **Split:** 363/91/115 amostras (64%/16%/20% treino/val/teste)
- Labels: **B=0, M=1** (invertemos o sklearn)
- Z-score: μ e σ só *do treino*
- **Treino:** labels em one-hot (B = [1, 0]; M = [0, 1]) para Softmax + CE
- **Seleção de hiperparâmetros:** só validação (16%)
- **Teste (20%):** avaliado uma única vez no final

BLOCO 1

Separação das classes · PCA 2D



TAKEAWAY

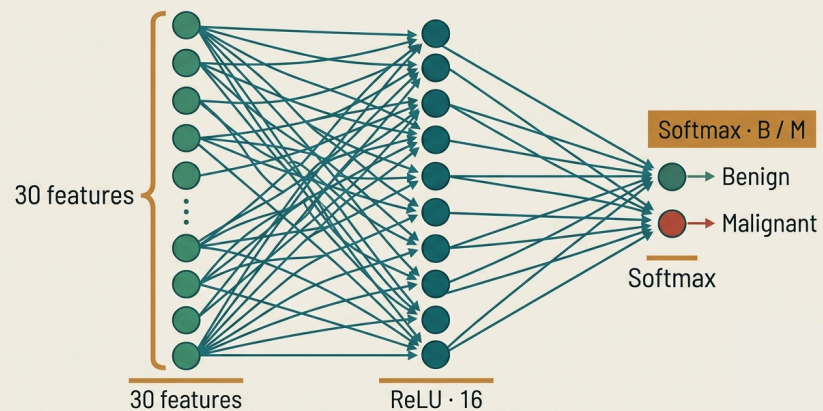
- Classes **parcialmente separáveis** mesmo em 2D
- Baseline já alinha $P(\text{maligno})$ com os clusters
- Overlap em $PC1 \approx 0$ a 2: ainda há casos difíceis (útil para interpretabilidade)

BLOCO 2

Arquitetura da MLP

Melhor config (val): [30, 16, 2]

Multilayer Perceptron Neural Network breast cancer tabular data



TAKEAWAY

- MLP : forward, backward, update, fit (gradient check ✓)
- 30 entradas tabulares, 16 neurônios ReLU, 2 logits Softmax
- Init default $N(0, 0.01)$; He nos experimentos extras

BLOCO 2

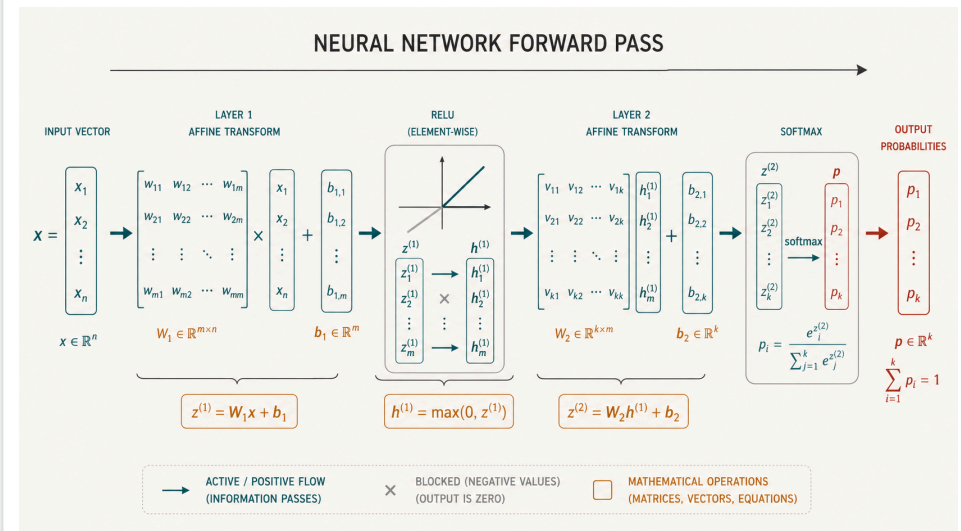
Forward pass

Oculto: $z^{(1)} = xW^{(1)} + b^{(1)}$, $h = \text{ReLU}(z^{(1)})$

ReLU: $\max(0, t)$ · backward: $1_{t>0}$

Saída: $z^{(2)} = hW^{(2)} + b^{(2)}$, $\hat{y} = \text{softmax}(z^{(2)})$

Softmax: $\hat{y}_k = e^{z_k} / \sum_j e^{z_j}$



► Dados fluem em frente; ReLU “desliga” neurônios com $z \leq 0$ (morte de ReLU no início do treino)

BLOCO 2

Loss · Softmax + cross-entropy

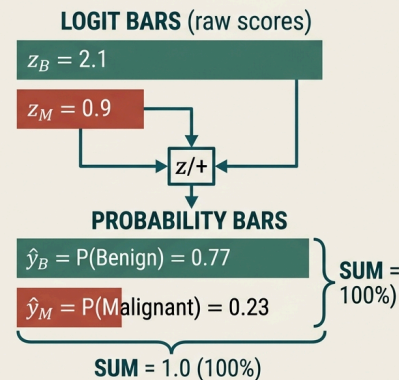
Cross-entropy (y one-hot, $\hat{y}$ softmax):

$$L = - \sum_k y_k \log \hat{y}_k$$

Gradiente na saída: $\partial L / \partial z^{(2)} = \hat{y} - y$

Mini-batch: média de L e ∇ sobre B amostras

SOFTMAX: FROM LOGITS TO PROBABILITIES



CROSS-ENTROPY LOSS: PRED VS. TRUE LABEL

CROSS-ENTROPY LOSS FORMULA

$$L = - \sum y_i * \log(\hat{y}_i)$$

Case 1: Actual is Benign (y = B)

$$y = \begin{bmatrix} 1 \\ 0 \end{bmatrix}^T, \hat{y} = \begin{bmatrix} 0.77 \\ 0.23 \end{bmatrix}^T \rightarrow \text{Loss } L = - (1 * \log(0.77) + 0 * \log(0.23)) = 0.26$$

Case 2: Actual is Malignant (y = M)

$$y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}^T, \hat{y} = \begin{bmatrix} 0.77 \\ 0.23 \end{bmatrix}^T \rightarrow \text{Loss } L = - (0 * \log(0.77) + 1 * \log(0.23)) = 1.47$$

Higher probability on true class => Lower Loss

[GRADIENT HINT: $L_{\text{gradient}} \propto \hat{y} - y$]

BLOCO 2

Backpropagation & update

Chain rule camada a camada

$$W \leftarrow W - \eta \cdot \nabla_W L$$

Momentum (extra): $v \leftarrow \mu v + \nabla W$, $W \leftarrow W - \eta v$

TAKEAWAY

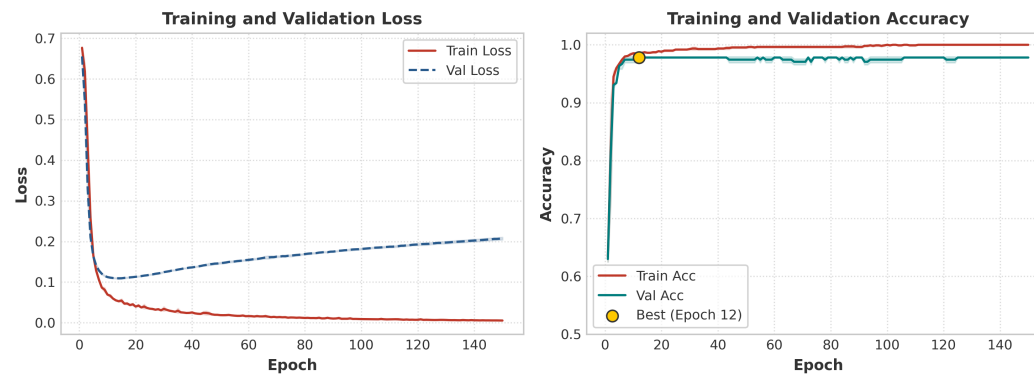
- Sem autograd: implementamos cada ∂ manualmente
- Exploramos $lr \in \{0.001, 0.01, 0.1\}$, $batch \in \{8, 32, 64\}$
- Jacobiano Softmax já “embutido” no $\hat{y} - y$ da CE

BLOCO 3

Baseline · treino inicial

arch=[30,32,2] · lr=0.1 · batch=32 · 150 ep · 3 seeds

Baseline Training History over 3 Seeds

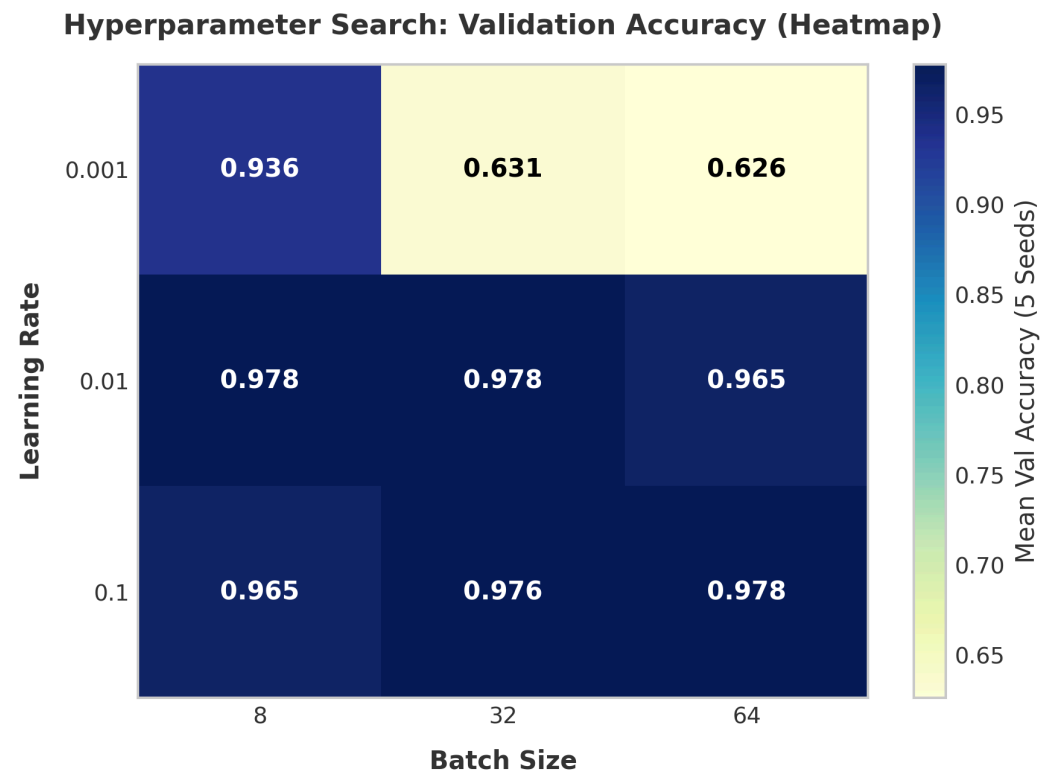


TAKEAWAY

- Converte rápido com lr alto, mas **val loss sobe** após ~época 15
- Val acc ~97 a 98%: bom ponto de partida, não o modelo final
- Marcador = melhor val acc (~época 12)

BLOCO 3

Grid · learning rate $\times$ batch size



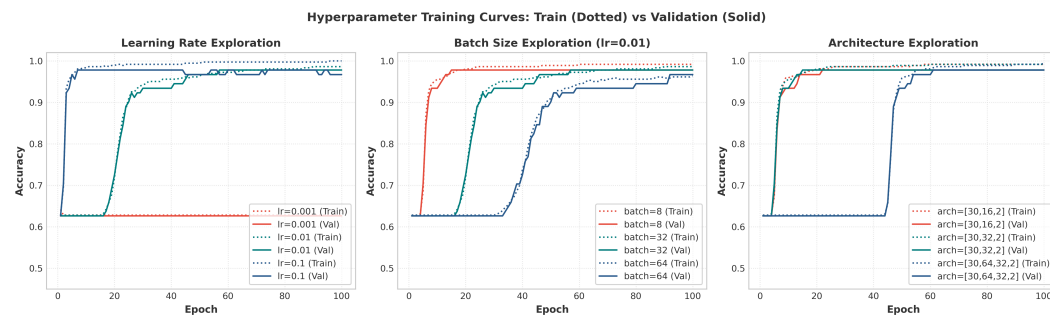
TAKEAWAY

- **lr=0.001** fica em ~0,63 (preso no majority baseline)
- Melhor célula: **lr=0.01, batch=8** com val acc 0,978
- Seleção sempre por **validation**, nunca por teste

BLOCO 3

Symmetry-breaking phase

Todas começam em ~62.7% = proporção da classe B

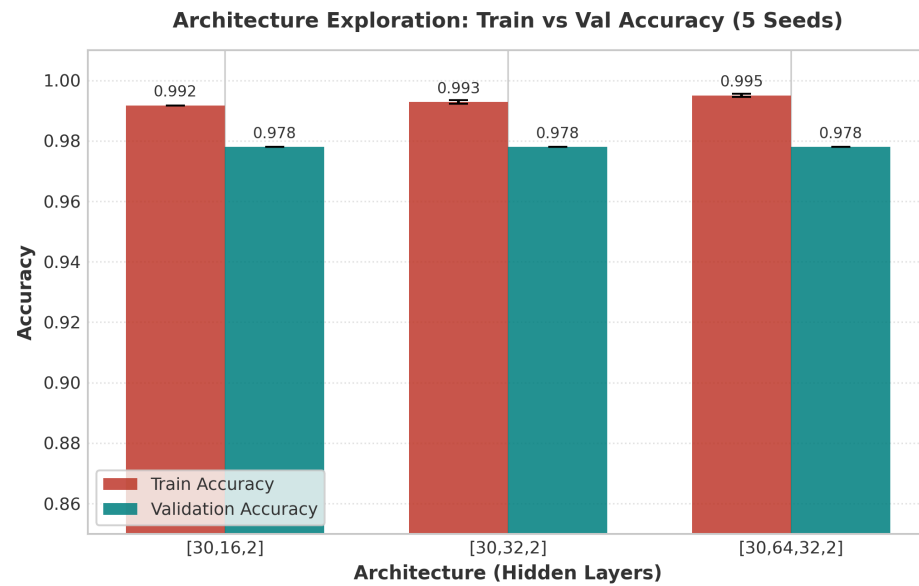


TAKEAWAY

- **Fase de latência:** pesos pequenos e ReLUs mortas mantêm o platô em 62,7%
- lr=0.1 quebra em ~2 ep; lr=0.01 em ~18 ep; lr=0.001 nunca sai
- Rede profunda [30,64,32,2] demora ~45 ep: mais camadas enfraquecem o gradiente

BLOCO 3

Arquitetura · shallow vs deep



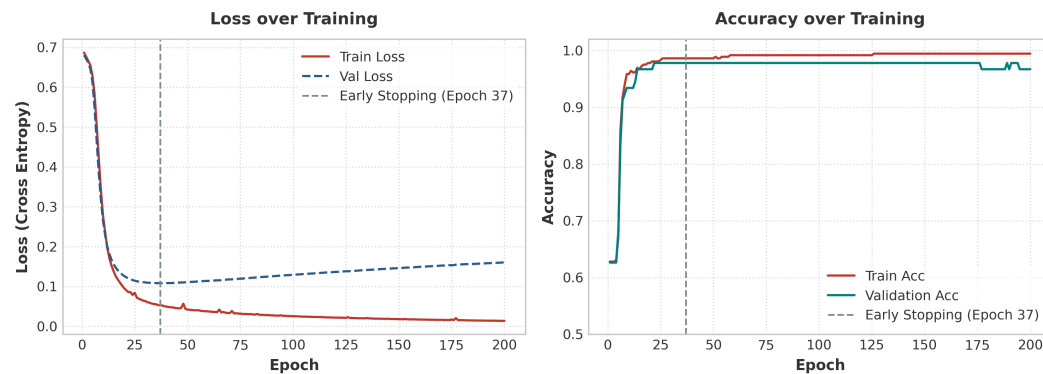
TAKEAWAY

- Val acc **0.978** nas 3: escolhemos **[30,16,2]** por parsimônia
- Rede maior: train sobe, val igual (overfitting sem ganho na generalização)
- **Config final:** lr=0.01, batch=8, arch=[30,16,2]

BLOCO 3

Early stopping · Stage 1

Stage 1: Train vs Validation (Optimal Stopping Epoch = 37)



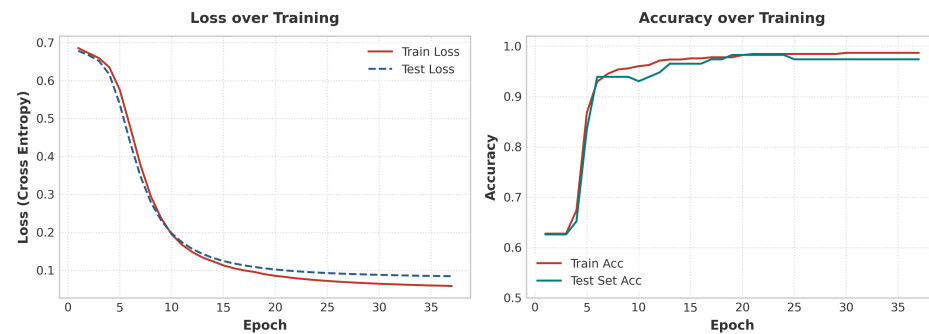
TAKEAWAY

- Época ótima $\approx$ **37** (mínimo val loss)
- Após isso: train melhora e **val piora** (overfitting)
- Define quantas épocas usar no retreino treino+val (Stage 2)

BLOCO 4

Retreino final · Stage 2

Stage 2: Final Model Training (Merged Train+Val, Trained for 37 Epochs)



97.4%
TEST ACC

37
ÉPOCAS

2+1
FP + FN

115
TEST N

BLOCO 4

Matriz de confusão · test set

Confusion Matrix - Final Model (Test Set)

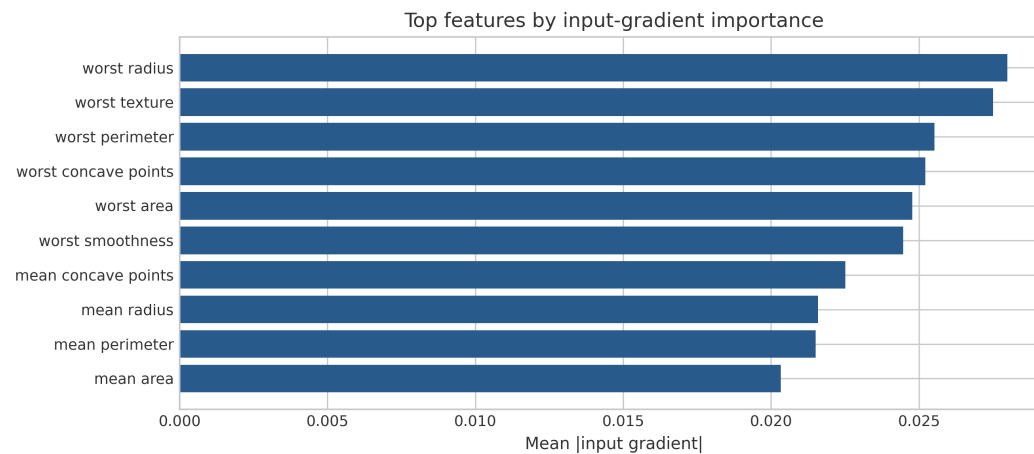
True Benign	70	2
True Malignant	1	42
	Predicted Benign	Predicted Malignant

TAKEAWAY

- **TN=70** benignos corretos · **TP=42** malignos corretos
- **2 FP** (alarme falso) · **1 FN** (maligno perdido)
- Erros raros, porém clinicamente relevantes (ver caso #39 na interpretabilidade)

BLOCO 4

Importância global · input gradients



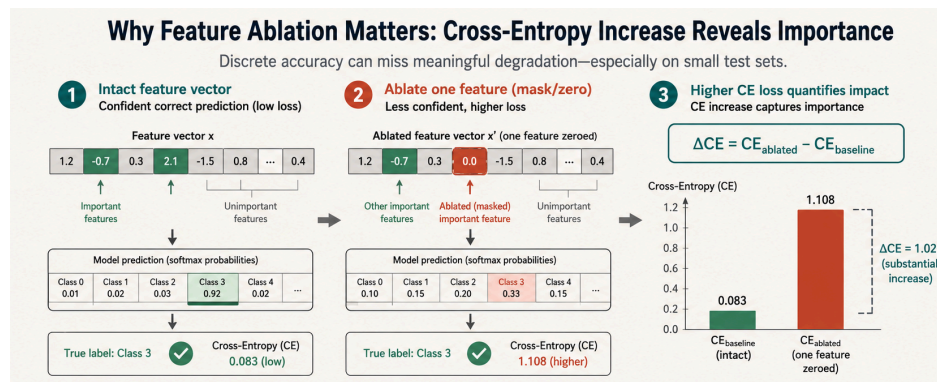
TAKEAWAY

- $|\partial L / \partial x_i|$ grande: pequena mudança em x_i altera muito a loss
- Top: **worst radius**, **worst texture**, concavidade
- Medida **local** (por amostra) agregada no test set

BLOCO 4

Por que ΔCE mede importância?

Ablação no código: zerar coluna(s) em `X_test_std` (sem embaralhar)



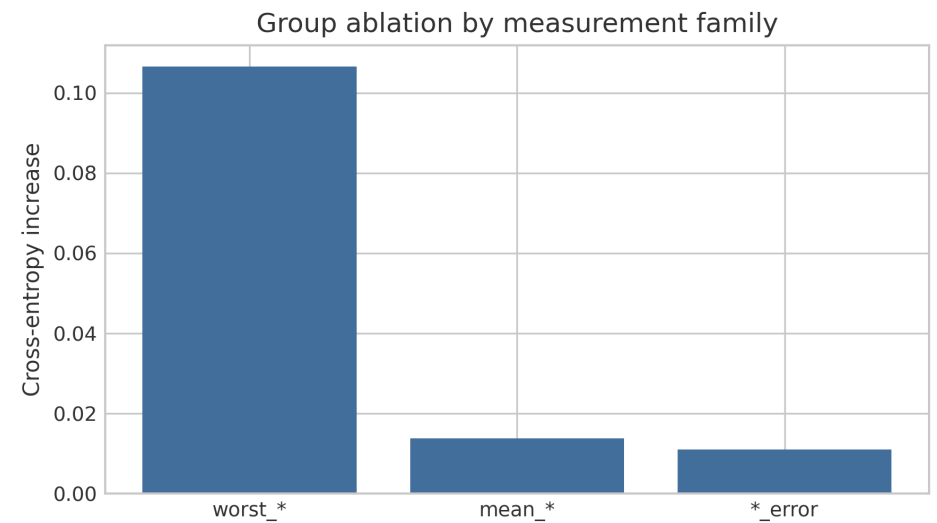
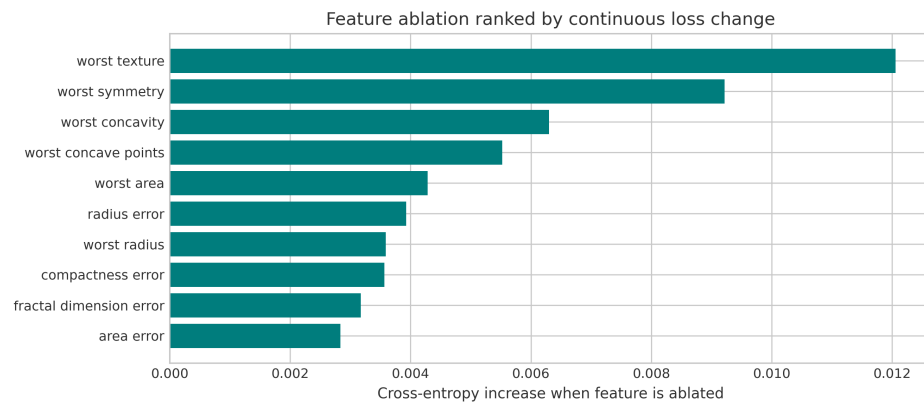
LÓGICA

- **Como ablamos:** copiamos o teste padronizado e fazemos `X[:, j] = 0` (uma feature ou um grupo)
- Com z-score (μ, σ do treino), **0 = média de treino** na escala original (“apaga” o sinal da feature)
- $\Delta CE = CE_{\text{ablated}} - CE_{\text{baseline}}$; quanto maior o ΔCE , mais importante a feature
- **Accuracy** ($n=115$) quase não muda (passos de ~0,87%); CE captura queda de confiança sem flip de classe

Não usamos permutação/embaralhamento de coluna

BLOCO 4

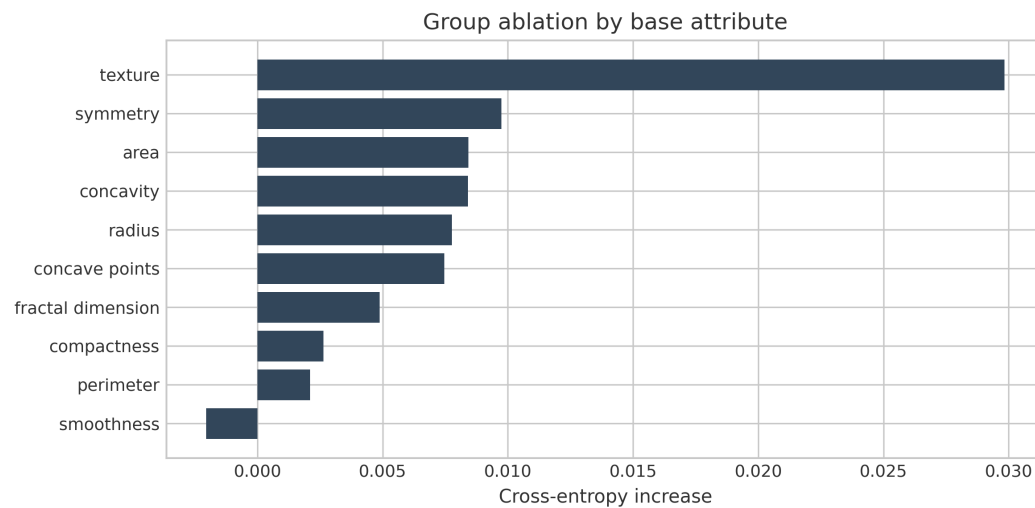
Ablação · resultados (ΔCE)



► Ablação: zerar em z-score ($0 = \mu_{\text{treino}}$) · família `worst_*` domina ($\Delta CE \approx 0,11$)

BLOCO 4

Ablação por atributo-base



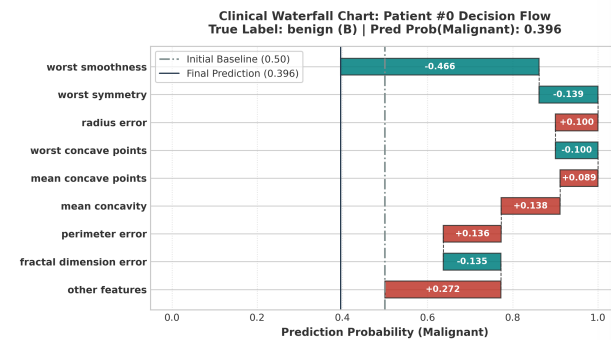
TAKEAWAY

- Zeramos as 3 colunas de cada atributo (`mean` , `error` , `worst`) de uma vez
- **texture** lidera em ΔCE , mas nem sempre coincide com o ranking por gradiente
- Confirma: medidas extremas (`worst_*`) e forma (concavidade) importam

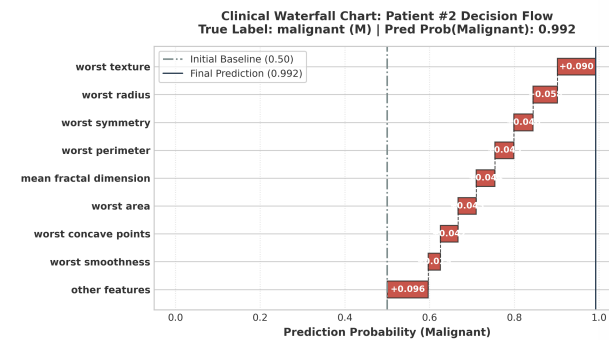
BLOCO 4

Casos individuais · gradient × input

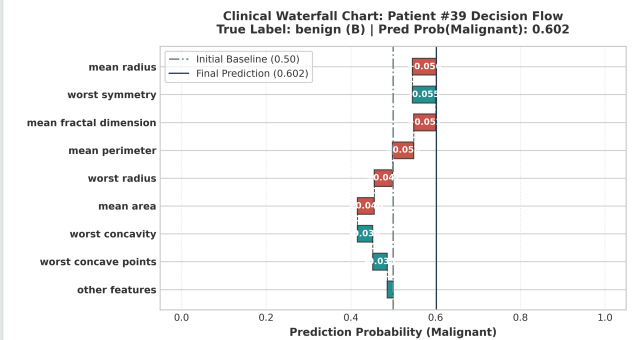
#0 · B → B · P(M)=0.40



#2 · M → M · P(M)=0.99



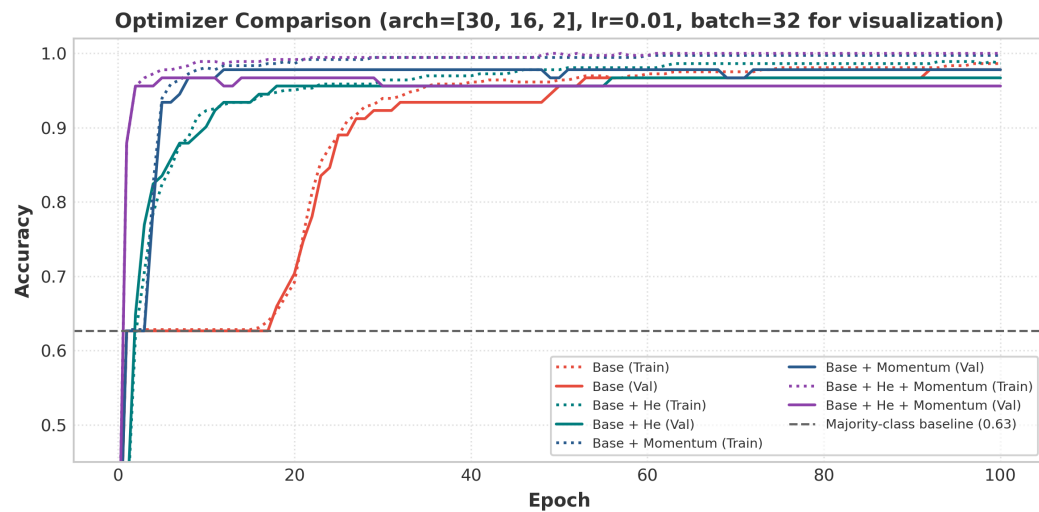
#39 · B → M · FP · P(M)=0.60



BLOCO 5

He initialization & momentum

Training Curves: Train (Dotted) vs Validation (Solid)
(batch=32 used here instead of best batch=8; epoch 0 = pre-training)



TAKEAWAY

- **batch=32** só para visualizar curvas (final usa batch=8)
- He + momentum quebram platô 62.7% em ~2 ep vs ~27 (base)
- Val final similar (~96 a 98%): o ganho é **velocidade**, não acurácia máxima

BLOCO 5

NumPy manual vs sklearn MLPClassifier

Nosso NumPy

- Init manual $N(0, 0.01)$
- **SGD** (Stochastic Gradient Descent): atualiza pesos com gradiente de mini-batches
- Backprop, Softmax e CE implementados à mão

sklearn MLPClassifier

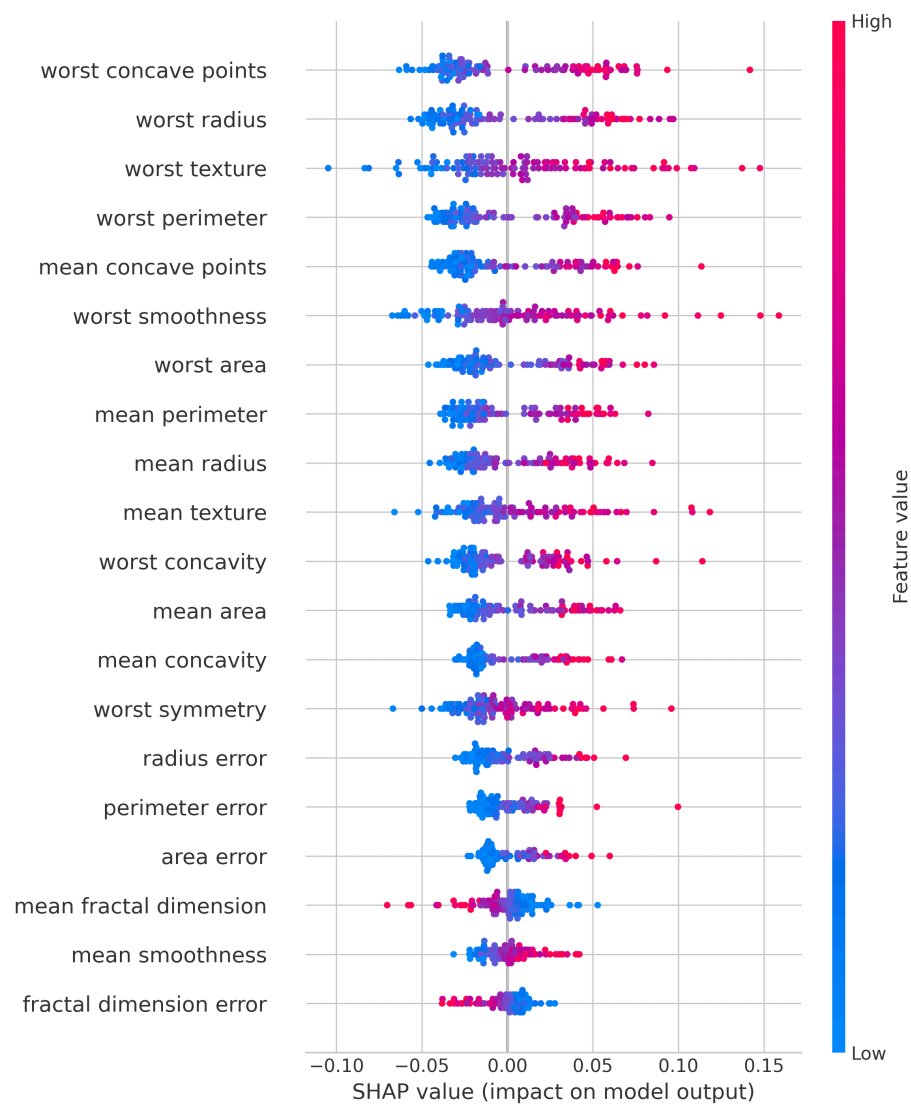
- Init **Glorot uniform** (padrão interno, diferente do nosso)
- Mesmo solver `sgd`, lr=0.01, batch=8, ReLU, 37 épocas
- Código otimizado em C; ordem numérica dos updates pode variar

MODEL	TRAIN ACC	TEST ACC
NumPy manual	0.987	0.974
sklearn	0.989	0.974

- Mesma arquitetura e hiperparâmetros; no **teste** as duas coincidem em **97,4%**, validando nosso código
- Train do sklearn ~0,2 pp acima: init **Glorot** e otimização interna em C
- Isso não indica melhor generalização, só diferenças de implementação numérica

BLOCO 5

SHAP · complemento



TAKEAWAY

- SHAP: impacto em P(maligno) por feature (modelo-agnóstico)
- Concorda em `worst_*`, radius, concavidade
- **Discordâncias** com gradiente/ablação são esperadas entre métodos diferentes

- **Test set pequeno** (115 amostras): acurácia em passos de ~0,87%; o resultado pode oscilar com poucas mudanças
- **Ablação por accuracy** quase não discrimina aqui; usamos ΔCE e queda de confiança
- **Discordância entre métodos:** gradientes, ablação e SHAP nem sempre ranqueiam as mesmas features
- **Protocolo:** hiperparâmetros na validação e teste usado uma vez; ainda há variância de sorte
- **Clínica:** features tabulares (anos 90), não imagem; não substitui avaliação médica completa

Obrigado

97.4%

TEST ACC

[30,16,2]

BEST ARCH

NumPy

ZERO DL LIBS